Experiment – 6

END-TO-END ML PIPELINE IMPLEMENTATION

Aim – To convert a machine learning workflow into a reusable, executable and reproducible end-to-end ML pipeline covering data preparation, preprocessing, feature engineering, model training, evaluation and prediction.

Task 1 — Dataset Preparation

- Select a suitable classification or regression dataset. You may continue with the dataset used in Experiment 5, or choose another meaningful dataset.
- Load the dataset using an appropriate Python library such as pandas.
- Inspect the dataset and identify the target variable, input features, data types and basic data-quality issues.
- Handle missing values, duplicate records and inconsistent data types where required.
- Identify numerical and categorical features and apply suitable preprocessing.
- Split the dataset into training and testing sets using an appropriate train/test ratio and a fixed random state for reproducibility.

Task 2 — Design the End-to-End ML Pipeline

- Design the complete workflow as a sequence of connected stages: Data Loading → Data Validation/Preparation → Preprocessing → Feature Engineering → Model Training → Model Evaluation → Prediction.
- Identify the input and output of every stage.
- Define the dependencies between stages so that the output of one stage becomes the input to the next stage.
- Ensure that preprocessing and feature transformations are fitted only on the training data and consistently applied to test/new data.
- Represent the workflow using a flowchart or pipeline diagram in the experiment report.

Task 3 — Modular Pipeline Implementation

- Convert the ML workflow into reusable Python modules/functions instead of executing separate notebook cells manually.
- Create separate logical components for data loading, preprocessing, feature engineering, model training, evaluation and prediction.
- Use appropriate configuration variables for dataset path, target column, test size, random state and model hyperparameters.
- Save the trained model and all preprocessing components required for inference, or use an appropriate scikit-learn Pipeline/ColumnTransformer.
- Use meaningful file names and maintain a clear project structure such as src/, data/, models/, outputs/ and config/.

Task 4 — Build and Execute the Pipeline

- Create a main pipeline script that executes the complete workflow from start to finish without requiring manual execution of individual notebook cells.
- Run the pipeline on the selected dataset.
- Verify that each stage executes successfully and produces the expected output.
- Record model evaluation metrics such as Accuracy, Precision, Recall, F1-score and ROC-AUC for classification, or suitable regression metrics such as MAE, MSE/RMSE and R^2.

- Generate and save important outputs such as the trained model, evaluation results and prediction file.
- Handle execution errors appropriately and record any problems encountered during the pipeline run.

Task 5 — Test Reproducibility and Prediction

- Run the complete pipeline again using the same dataset, configuration and random state.
- Compare the results of the repeated run and verify whether the workflow produces consistent or explainable results.
- Load the saved model/pipeline and perform prediction on unseen test or new input data.
- Verify that the prediction stage uses exactly the same preprocessing and feature representation used during training.
- Record the final predictions and model performance in the experiment report.

Task 6 — Documentation and Report

- Document the complete pipeline architecture and explain the purpose of every stage.
- Include the dataset description, preprocessing operations, feature engineering steps, model selected and hyperparameters.
- Include the evaluation metrics and explain the obtained results.
- Include the project folder structure and important code/output screenshots.
- Record the observations, errors encountered, corrective actions and final outcome.

Expected Deliverables

- Dataset and preprocessing code
- Modular ML pipeline implementation
- Pipeline flowchart/architecture diagram
- Data preprocessing and feature-engineering modules
- Training and evaluation modules
- Saved trained model and preprocessing pipeline
- Prediction output
- Evaluation metrics/results
- Pipeline execution screenshots
- Reproducibility verification
- Short experiment report

Tips

- You can use the same dataset/model scenario from Experiment 5, such as CKD prediction, or choose another meaningful classification/regression problem.
- Do not keep the complete workflow dependent on manually running notebook cells; the objective is to create an executable pipeline.
- Keep training and inference preprocessing identical. A scikit-learn Pipeline with ColumnTransformer is recommended where applicable.
- Use a fixed random_state when appropriate so that repeated executions can be compared.
- Keep data, source code, models and generated outputs organized in separate directories.
- Avoid hard-coded absolute paths. Use configuration variables or relative project paths.
- Save the final model together with every transformation required to process new data.
- For a stronger implementation, add data validation, logging, exception handling and MLflow experiment tracking.
- A good submission should demonstrate the complete flow: raw dataset → preprocessing → feature engineering → training → evaluation → saved model → prediction.

Suggested Project Structure

- project/
- ├── data/ # dataset
- ├── src/ # pipeline source code
- │ ├── data_loader.py
- │ ├── preprocessing.py
- │ ├── feature_engineering.py
- │ ├── train.py
- │ ├── evaluate.py
- │ └── predict.py
- ├── models/ # saved model/pipeline
- ├── outputs/ # metrics and predictions
- ├── config/ # configuration files
- └── main.py # end-to-end pipeline entry point

TOOLS THAT CAN BE USED:-

- Docker —containerization of the complete pipeline.
- DVC —dataset/model versioning for reproducible ML workflows.
- Git / GitHub or GitLab — source-code version control and collaboration.
- MLflow —experiment tracking, metric logging and model management.
- Pytest —unit and pipeline testing.
- YAML / JSON — configuration of paths, parameters and pipeline settings.
- Joblib or Pickle — saving and loading trained models and preprocessing objects.
- Matplotlib / Seaborn —visualization of data and model results.
- Scikit-learn — preprocessing, ColumnTransformer/Pipeline, model training and evaluation.
- Pandas / NumPy — data loading, cleaning and numerical processing.
- Jupyter Notebook / Google Colab —exploration and initial experimentation.
- Python — implementation of the end-to-end ML pipeline.